

## Table of Contents

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| Classe Response con Pattern Builder.....                                | 2  |
| 1.1 Panoramica.....                                                     | 2  |
| 1.2 Architettura.....                                                   | 2  |
| 1.3 Struttura delle Classi.....                                         | 3  |
| Classe Response (Product).....                                          | 3  |
| Classe ResponseBuilder (Builder).....                                   | 3  |
| 1.4 Formato di Output JSON.....                                         | 3  |
| 1.5 Esempi di Utilizzo.....                                             | 4  |
| Esempio 1: Risposta di Successo di Base.....                            | 4  |
| Esempio 2: Risposta di Errore con Codice.....                           | 4  |
| Esempio 3: Risposta con Dati Complessi.....                             | 4  |
| Esempio 4: Utilizzo Diretto del Costruttore (versione alternativa)..... | 4  |
| 1.6 Vantaggi del Pattern Builder.....                                   | 5  |
| 1.7 Gestione degli Errori.....                                          | 5  |
| 1.8 Strategia di Testing.....                                           | 6  |
| 1.9 Dettagli Implementazione Test.....                                  | 6  |
| 1.10 Messaggi di Errore.....                                            | 7  |
| 1.11 Esempi di Dati nei Test.....                                       | 7  |
| 1.12 Flusso Completo di Creazione.....                                  | 9  |
| 1.13 Decisioni Chiave nella Progettazione.....                          | 9  |
| 1.14 Esecuzione dei Test.....                                           | 10 |
| 1.15 Risultati Attesi dai Test.....                                     | 10 |

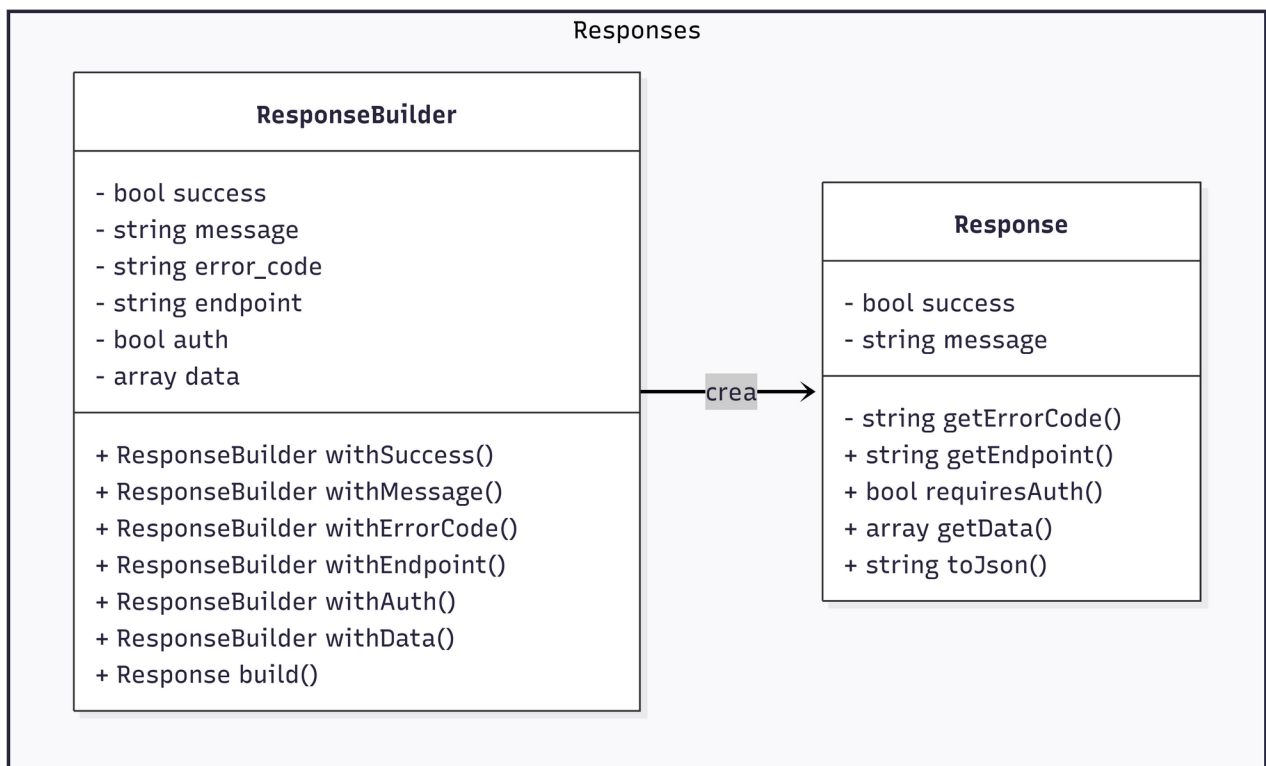
# Documentazione del Sistema di Routing API - Componente Response

## Classe Response con Pattern Builder

### 1.1 Panoramica

Il componente **Response** implementa il **Design Pattern Builder** per creare oggetti complessi di risposta API in modo strutturato e mantenibile. Questo pattern separa la costruzione di un oggetto complesso dalla sua rappresentazione, permettendo lo stesso processo di costruzione per creare rappresentazioni diverse.

### 1.2 Architettura



## 1.3 Struttura delle Classi

### *Classe Response (Product)*

La classe **Response** rappresenta l'oggetto finale di risposta API con le seguenti proprietà:

| Proprietà'   | Tipo          | Descrizione                                        | Richiesto |
|--------------|---------------|----------------------------------------------------|-----------|
| \$success    | bool          | Indica se l'operazione ha avuto successo           | si        |
| \$message    | string        | Messaggio leggibile che descrive il risultato      | si        |
| \$error_code | string   null | Codice di errore per operazioni fallite (nullable) | no        |
| \$endpoint   | string        | Endpoint API chiamato                              | si        |
| \$auth       | bool          | Indica se l'autenticazione e' richiesta            | si        |
| \$data       | array         | Dati aggiuntivi (opzionali)                        | no        |

### *Classe ResponseBuilder (Builder)*

La classe **ResponseBuilder** fornisce un'interfaccia per costruire oggetti **Response**:

| Metodo          | Parametri                  | Tipo ritornato  | Descrizione                            |
|-----------------|----------------------------|-----------------|----------------------------------------|
| withSuccess()   | bool \$success             | ResponseBuilder | Imposta lo stato di successo           |
| withMessage()   | string \$message           | ResponseBuilder | Imposta il messaggio                   |
| withErrorCode() | string   null \$error_code | ResponseBuilder | Imposta il codice di errore            |
| withEndpoint()  | string \$endpoint          | ResponseBuilder | Imposta l'endpoint                     |
| withAuth()      | bool \$auth                | ResponseBuilder | Imposta il requisito di autenticazione |
| withData()      | array \$data               | ResponseBuilder | Imposta dati aggiuntivi                |
| Build()         | nessuno                    | Response        | Crea e valida la Response              |

## 1.4 Formato di Output JSON

Il metodo `convertToJson()` ritorna un JSON con una struttura simile:

```
{
  "success": true,
  "message": "Operazione completata con successo",
  "endpoint": "/api/operazione",
  "auth": false,
  "error_code": "CODICE_ERRORE",
  "data": {
    "key_1": "val_1",
    "key_2": "val_2"
  }
}
```

## 1.5 Esempi di Utilizzo

### ***Esempio 1: Risposta di Successo di Base***

```
$response = (new ResponseBuilder())
->withSuccess(true)
->withMessage("Login effettuato con successo")
->withEndpoint("/api/login")
->withAuth(false)
->build();

echo $response->convertToJson();
```

### ***Esempio 2: Risposta di Errore con Codice***

```
$response = (new ResponseBuilder())
->withSuccess(false)
->withMessage("Autenticazione fallita")
->withEndpoint("/api/login")
->withErrorCode("AUTH_FAILED")
->withAuth(true)
->build();
```

### ***Esempio 3: Risposta con Dati Complessi***

```
$datiUtente = [
    "id" => 123,
    "nome" => "Mario Rossi",
    "email" => "mario@esempio.com"
];

$response = (new ResponseBuilder())
->withSuccess(true)
->withMessage("Profilo utente caricato")
->withEndpoint("/api/utente/profilo")
->withData($datiUtente)
->build();
```

### ***Esempio 4: Utilizzo Diretto del Costruttore (versione alternativa)***

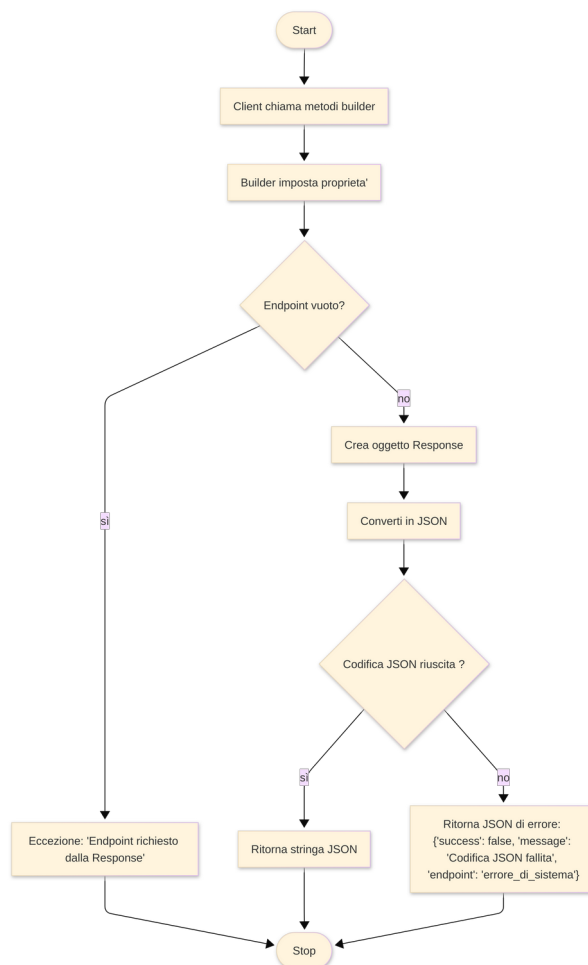
```
$response = new Response(
    true, // success
    "Operazione completata con successo", // message
    null, // error_code (null per successo)
    "/api/test", // endpoint
    false, // auth
    ["risultato" => "ok"] // data
);
```

## 1.6 Vantaggi del Pattern Builder

| Vantaggio                | Descrizione                                          | Implementazione                            |
|--------------------------|------------------------------------------------------|--------------------------------------------|
| Costruzione step-by-step | Oggetti complessi vengono costruiti incrementalmente | Ogni metodo with*() aggiunge una proprietà |
| Validazione              | L'oggetto viene validato prima della creazione       | build() valida i campi richiesti           |
| Immutabilità             | Gli oggetti sono immutabili dopo la creazione        | Non ci sono setter nella classe Response   |
| Interfaccia consistente  | Stesso processo di costruzione per tutte le risposte | I metodi del builder sono uniformi         |
| Parametri opzionali      | Gestisce i campi opzionali in maniera semplice       | Solo i campi richiesti vengono validati    |
| Vantaggio                | Descrizione                                          | Implementazione                            |

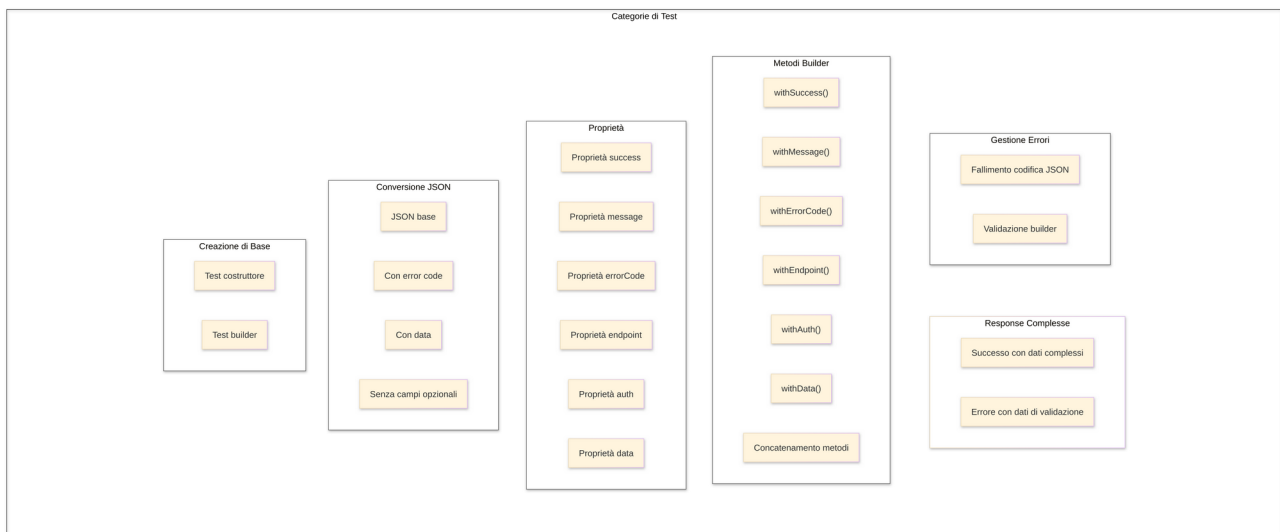
## 1.7 Gestione degli Errori

Il componente Response include una robusta gestione degli errori:



## 1.8 Strategia di Testing

La classe ResponseTest implementa il testing completo:



## 1.9 Dettagli Implementazione Test

La classe ResponseTest include 7 sezioni principali di test:

- 1 **Test della creazione di una Response di base** - Test del costruttore e inizializzazione del builder
- 2 **Test dei metodi con design pattern Builder** - Test di ogni metodo builder e dei metodi concatenati
- 3 **Test delle proprietà di Response** - Test dei metodi getter per tutte le proprietà
- 4 **Test della conversione in JSON** - Test della serializzazione JSON con varie configurazioni
- 5 **Test della gestione degli errori** - Test degli scenari di fallimento della codifica JSON
- 6 **Test di validazione di Builder** - Test della validazione dell'endpoint nel builder
- 7 **Test su Response complesse** - Test di scenari realistici di risposta API

Ogni test include:

- Descrizioni chiare dei test
- Testing di asserzioni singole e multiple
- Reporting di successo/fallimento tramite stile CSS
- Gestione appropriata delle eccezioni

## 1.10 Messaggi di Errore

Il sistema include messaggi di errore:

```
// Nel metodo build() di ResponseBuilder
if (empty($this->endpoint)) {
    throw new Exception('Endpoint richiesto dalla Response');
}

// Nel metodo convertToJson() di Response
return json_encode([
    'success' => false,
    'message' => 'codifica JSON fallita',
    'endpoint' => 'system_error'
]);
```

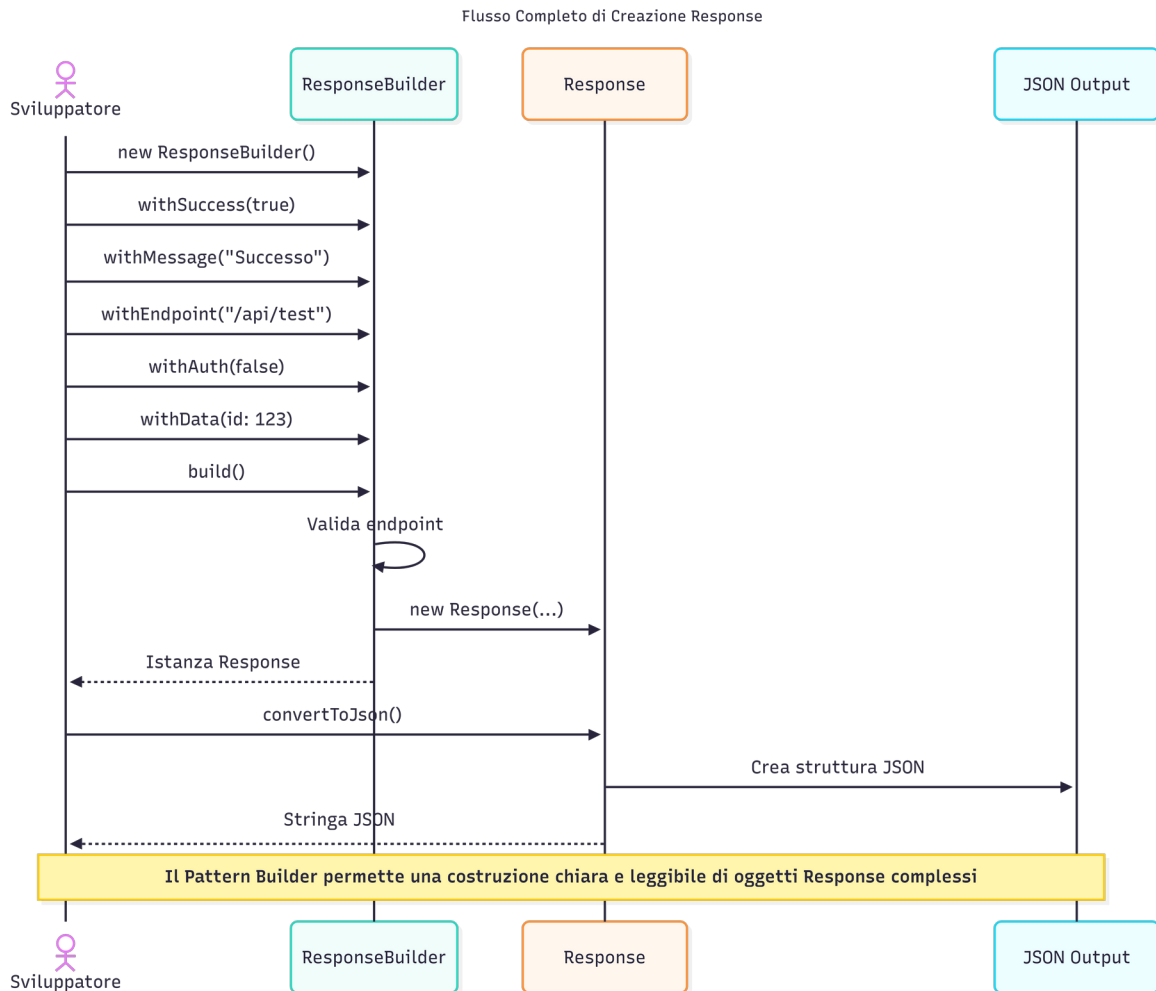
## 1.11 Esempi di Dati nei Test

Test complessi:

```
$datiComplessi = [
    "utente" => [
        "id" => 123,
        "nome" => "Pinco Pallino",
        "email" => "pinco@esempio.com",
        "ruoli" => ["utente", "amministratore"]
    ],
    "token" => "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9",
    "scade_in" => 3600
];

$response = (new ResponseBuilder())
    ->withSuccess(true)
    ->withMessage("Utente autenticato con successo")
    ->withEndpoint("/api/login")
    ->withAuth(true)
    ->withData($datiComplessi)
    ->build();
```

## 1.12 Flusso Completo di Creazione



## 1.13 Decisioni Chiave nella Progettazione

- 1 **Implementazione in un Singolo File:** Sia *Response* che *ResponseBuilder* sono in un singolo file per mantenere l'integrità del pattern ed evitare problemi di scope PHP.
- 2 **Interfaccia Consistente:** Tutti i metodi builder ritornano *\$this* per permettere il concatenamento dei metodi, migliorando la leggibilità e quindi la revisione del codice.
- 3 **Validazione Endpoint:** Il builder valida che l'endpoint non sia vuoto prima di creare la Response, garantendo l'integrità dei dati.
- 4 **Gestione Errori JSON:** Gestione dei fallimenti di codifica JSON con una risposta di errore di fallback.
- 5 **Design Immutabile:** La classe Response non ha metodi setter, garantendo che gli oggetti siano immutabili dopo la loro creazione.



## 1.14 Esecuzione dei Test

Per eseguire i test Response, una volta lanciata l'applicazione con Docker:

```
docker compose up --force-recreate --remove-orphans -d
```

Bastera' navigare presso la pagina <http://localhost:8080/test/> e cliccare il link [Test Page for Response](#).

## 1.15 Risultati Attesi dai Test

Ogni test produrra' output simile a:

Test della classe Response con design pattern Builder

### Test della creazione di una Response di base

- Creazione della Response tramite costruttore -

**PASSED:** `assertInstanceOf(Response)`

- Creazione della response tramite builder -

**PASSED:** `assertInstanceOf(Response)`

*Con CSS styling che evidenzia:*

- **Successo:** Contorno verde

- **Fallimento:** Contorno rosso